MoeCTF 2026 Crypto

MoeCTF 2026 Crypto 方向 出题人 WP

题目顺序与平台一致

密码学入门指北

分数：49 pts

本题背景为Diffie-Hellman密钥协商算法，请注意，这个算法主要解决的是密钥配送问题而非加密问题。

具体的场景是：对称加密需要发送者和接受者使用同一个密钥，因此需要事先协商好密钥。但是如果密钥在传输过程中产生泄露，就会出现严重的问题。

DH算法简介：

1. Alice与Bob共享一个素数p以及对应的原根 g $(2 \leq g \leq p-1)$ 。值得一提的是，这两个数可以不经加密地由一方发送到另一方。
2. Alice生成一个私钥A($1 \leq A \leq p-2$)，然后计算公钥 $g^A = Y_a \mod p$,将公钥 Y_a 通过公网发给Bob。同理，Bob也生成私钥 B ,公钥 Y_b ,将 Y_b 发送给Alice。在这个过程中，敌手可以截获公钥 Y_a, Y_b 但不能恢复共享钥匙 key 。
3. 此时，Alice可以计算 $Y_b^A = key$,Bob可以计算 $Y_a^B = key$ 。这样就在不发送私钥的情况下共享了密钥。

回到入门指北给的这道题，可以看到在打印公钥的同时也错误地打印了两人的私钥，只要用其中一个人的私钥和另一个人公钥计算得出key，就可以解密。

代码块

```
p = 170141183460469231731687303715884105727
g = 3
bob_public = 101533001028006636636416596392258549313
alice_private = 20260704
ciphertext = [45, 47, 37, 35, 52, 38, 59, 36, 40, 31, 48, 50, 41, 54, 33, 52,
37, 31, 43, 37, 57, 31, 51, 40, 47, 53, 44, 36, 31, 51, 52, 33, 57, 31, 51,
37, 35, 50, 37, 52, 61]

key = pow(bob_public,alice_private,p) % 256
flag = bytes(x ^ key for x in ciphertext)
```

```
print(flag.decode())
```

moeSign1n

出题人：hata　分数：50 pts

本题考察了 RSA 这样的一个性质 (同态性)：设 m_0 的密文 $c_0 \equiv m_0^e \pmod{n}$

若 $m_0 = m_1 \cdot m_2$ ， $c_1 \equiv m_1^e \pmod{n}$ ， $c_2 \equiv m_2^e \pmod{n}$

则 $c_0 \equiv c_1 \cdot c_2 \pmod{n}$ ，这一点很容易验证

故只需要简单计算

代码块

```
from Crypto.Util.number import *

m0 = bytes_to_long(b'MoeCTF 2026') # 936133387626562244155930l4
m1 = 2
m2 = m0 // 2
print(long_to_bytes(m1).hex())
print(long_to_bytes(m2).hex())
# 02
# 26b7b2a1aa23101918191b
```

先选择选项1，提交 02 和 26b7b2a1aa23101918191b，得到两个密文，计算他们的乘积，然后选择选项2提交即可获得 flag `moectf{0p3n_4_d00r_70_7h3_w0r1d_0f_m0d3rn_cryp706r4phy}`

wsl_Sign1n

出题人：youzi　分数：50 pts

在wsl下的sage环境中运行附件即可

justXOR

出题人：hata　分数：100 pts

最后是一个异或加密，密钥由一个数列的递推公式产生，但是直接递推计算时间不可行；由递推公式求出通项公式，可以快速计算；实际上就是这样一个式子

$$a_n \equiv 3a_{n-1} + 2 \pmod{M}$$

如果没有模 M，只是一个等式的话，相信大家肯定都会算 (高中知识)；不过实际上有模 M 也相差不大，照猫画虎即可，这里并不需要太多模运算的东西

$a_1 = 1$，当 $n \geq 2$ 时，有 $a_n \equiv 3a_{n-1} + 2 \pmod{M}$

设 $a_n + t \equiv 3(a_{n-1} + t) \pmod{M}$，则 $a_n \equiv 3a_{n-1} + 2t \pmod{M}$

$\therefore t = 1$，于是 $a_n + 1 \equiv 3(a_{n-1} + 1) \pmod{M}$

这是一个等比数列的形式，故

$$a_n + 1 \equiv 3(a_{n-1} + 1) \equiv 3^2(a_{n-2} + 1) \equiv \cdots \equiv 3^{n-1}(a_1 + 1) \pmod{M}$$

最终得到

$$a_n \equiv 2 \cdot 3^{n-1} - 1 \pmod{M}$$

exp 如下

代码块

```
M = 203912963320800909041490121230423409162623392392330104239841648912371906508106577603312756187603312792447 1
n = 10**25
key = (2 * pow(3, n - 1, M) - 1) % M
enc = b'it\x0bM\xfa-\xe3T{\xaa\x0f@\xa2\xf1@\xbd\x86e\x85\x9e\xfcp_o\x8f\xccd\x13\xceW\x13\x14\x11\x055b\xcbk\xa0'
from Crypto.Util.number import long_to_bytes
flag = bytes([x ^ y for x, y in zip(enc, long_to_bytes(key))])
print(flag)
# moectf{a_simple_sequence_problem_for_u}
```

the_KEY_of_DES

出题人：hata　分数：100 pts

DES 的有效密钥是 56bit，但是会添加 8bit 的校验位，最终变成64bit；校验位与加解密无关

`from Crypto.Cipher import DES` 后，需要给 DES 提供一个 64bit 的密钥，但是并不会进行是否正确的检查，适当修改校验位即可实现”碰撞”

`from Crypto.Cipher import DES` 的处理是对于 56bit 的有效密钥，分为 8 组，每组 7bit，在每组的末端加入校验位；故只改变一个组的末尾即可，一个构造如下

代码块

```
56bit: \x00\x00\x00\x00\x00\x00\x00
key1: 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
```

```
key2: 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000001
```

也就是说提交下面两个密钥即可获得 flag
`moectf{7h3_l177l3_7r1ck_w17h_7h3_ch3ck_d1g17}`

代码块

```
0000000000000000
0000000000000001
```

1391 超市

出题人：hata　分数：150 pts

这个题的初衷就是想搞一下时间上的侧信道，原理也非常简单，如果服务器端在校验一个东西的时候是逐个字节去比对，遇到不一致的就结束；那么就可以逐个字节去爆破，通过耗时来判断当前字节是否正确，显然正确的字节会进入下一轮比较，因而耗时会更长

故为了放大单字节比较的时间，使之能体现出差异，设计了 1391 次迭代哈希计算 (~~致敬 1931~~)

其实本来是打算出成远程题目让选手自己去搞的，但是我本地通了，远程不通 (有延迟等干扰因素)，最后也没有调好，故就以这种形式放出来了。事实上直接看 output.txt 也能发现每个字节总有一个值是异常的大，直接提取出来拼起来就是 flag `moectf{abed11be2e115680f30bd4f0d2}`

> 最后一个字节形式有所不同，大家可以看看代码，想一想这是为什么

这里放一个本地测试的脚本，感兴趣的话可以自己玩一玩

代码块

```python
from pwn import *
import time

io = process(["python", "-u", "./chal.py"])
io.recvuntil(b'choice: ')
name = '4e61694c6f6e67204d6f64656c'

def attack_one_byte(index):
    times = []
    for i in range(256):
        tmp = hex(i)[2:].zfill(2)
        ticket = name + ', ' + known + tmp + '00' * (12 - index)

        start = time.perf_counter()
        for _ in range(70):
            io.sendline(b'1')
```

```
            io.recvuntil(b'ticket: ')
            io.sendline(ticket.encode())
            io.recvuntil(b'choice: ')
        end = time.perf_counter()
        # print((end - start, tmp))
        times.append((end - start, tmp))

    times.sort(reverse=True)
    return times[0]

known = ''
for i in range(12):
    tmp = attack_one_byte(i)[1]
    print(f'{i}-th byte: {tmp}')
    known += tmp

for i in range(256):
    last = hex(i)[2:].zfill(2)
    io.sendline(b'1')
    io.recvuntil(b'ticket: ')
    io.sendline((name + ', ' + known + last).encode())
    tmp = io.recvline()
    correct = False
    if b'We will' in tmp:
        correct = True
        print(io.recvline())
        # break
    io.recvuntil(b'choice: ')
```

ez_f3mr4t

出题人: starwalking　分数：150pts

一个普通的rsa加密，但是质数的生成有漏洞，p是通过正常的 `getStrongPrime` 生成的，但 q 的方式依赖于 p， `q = next_prime(p ^ ((1<<512)-1))`

我们不妨记 `A = (1 << 512) - 1`，那么我们有如下算式

$$p \oplus A = A - p$$

也即 `q = next_prime(A - p)` 由于素数的分布并不是特别稀疏，我们可以认为q 和 A-p后的下一个素数是差不多的，也就是

$$q \approx A - p$$
$$p + q \approx A$$
$$S = p + q = A + \epsilon$$

利用费马分解法，我们这里可以快速分解n

我们不妨做如下变换

$$
\begin{aligned}
&设p = a + b \\
&\quad q = a - b \\
&则n = pq = a^2 - b^2 \\
&且a = \frac{p+q}{2}
\end{aligned}
$$

由于 p + q的近似值已知，那么可以知道 a 的近似值，枚举 a 我们判断 $n - a^2$ 是否为平方数即可知道 b，那么依据 p = a + b , q = a - b即可成功获取 p，q

解题代码如下

代码块

```
from gmpy2 import isqrt
from Cryptodome.Util.number import *

n = 0x308244e7a7de386723c92ba62e35bc22c3ec1b93023e1551408344a4ba31c6203da849aedee0cadf26a3442f9fd652f7d97c053a3f2c298eab6c0f0c2d0b4642a81765ddb00b690425eb212d5520327ac2d53a22922448399fecb54fbc04dbb68fa33fee7666cb9e05278b5f5f1330b3918d3a7def580fcc00f6f596f16eba3b
c = 0x13a77e34f09b8a2291cdb397ea0e5cb9e86f691c6565b35c76e6bb6248b67db24315e22fe1dc5321a8820320cdd9b51e3d431459aac2213948f4fbf01c4ce974428d1ed745b2c06f8aa92b22dfede3b7ceb59aa4eb22467129f55b60037a1a2de9b37f25fda3fe40323fccc7c8bbdd4446096b37cef4138337ee9a04c2e3d5fd
e = 0x10001
A = (1 << 512) - 1
a = A // 2 - 1

while True:
    b2 = a * a - n
    b = isqrt(b2)
    if b * b == b2:
        break

    a += 1

p = a + b
q = a - b
assert p * q == n

phi = (p - 1) * (q - 1)
d = pow(e,-1,phi)
m = pow(c,d,n)
```

```
flag = long_to_bytes(m).decode()
print(flag)
```

ez_fermat

出题人：youzi　分数：150 pts

可以看到是一个正常的rsa加密，只是额外给出了 `hint` ：

$$hint = (ap + b)^q \mod n$$

注意到，可以通过改变模数让 `hint` 的表达式更简洁，我们将 n 改为 p：

$$hint = b^q \mod p$$

同时，由费马小定理得

$$b^p = b \mod p$$

$$b^{pq} = b^n = b^q \mod p$$

因此，我们可整理出一个只有未知量p的式子

$$b^n = hint \mod p$$

然后就是经典套路

$$(n, b^n - hint) = p$$

代码块

```
import math
from math import gcd

n = 
15962603324053600624662899467930954606237037554051572101066176482327650464579501876010780067612122315010349894395035821617153081982429716234378979239633769779503395609504372912298014522344609346937078959678893005192469085672940628091531228571447786013091492389216237976408692434723698967667324605921896928717731731591488800520224411239128106892402734198486634978989319563704879076976307439748456769245703782902660583833942684475506024284563786886608632662629581206716450648212025254862836883526055933841863348546872473877054469843468332542834452213365868510001991952142536759384940216228753729399187149594010090304067
c = 
42819190684248860122144134359573796355174660436473801763502212536146239905994524959386897580012560453200614913763557501640513267349279059722673856115528948381400098984502674536530974662868384608899688960322165938119849425111652281430894579886304358103046724541471495561105422030843419015985195876110968934323072365589702645367196890393180235726801852468716009893179923995367297179580726422438346890619286679134958846820292120150716137304018432716411972540899004892317385188 7
```

```
641526200168700201297266566236494045767804382976812626321129525846520241968628199774986118004143940786619763636952519297192666055921757371393
hint =
889436479028069355612445766677650161402350947592716401649785862922921540912896502067464206359277093505720971998929454826383392747797226726795121418420904616877319634953792698212338247352203951586213875540097835012812823370179575815214327266894562611994937616234474976049916521022441275196432215003719940111125886943967276288379205181811250003219213194864455768662175986762944082469176839588279793661683957290684125100068105428316449741931493836387113925328913445846650169823503923445515497516511129619820909158655099087320953425692940068901703461880723250874744907648685278619934070441912053852620869106689803536229607
b = 0x2026

p = gcd(pow(b, n, n) - hint, n)
q = n // p

e = 65537
phi = (p - 1) * (q - 1)
d = pow(e, -1, phi)
m = pow(c, d, n)

flag = m.to_bytes((m.bit_length() + 7) // 8, "big")
print("flag =", flag.decode())
```

mid_det

出题人：hata　分数：150 pts

本题顺延了去年的 ez_det，再出个行列式玩一玩；实际上需要求解的就是这个循环矩阵的行列式，先考虑没有加上 a 时的情况

$$\begin{pmatrix} 1 & 2 & 3 & \cdots & n \\ n & 1 & 2 & \cdots & n-1 \\ n-1 & n & 1 & \cdots & n-2 \\ \vdots & \vdots & \vdots & \cdots & \vdots \\ 2 & 3 & 4 & \cdots & 1 \end{pmatrix}$$

有许多方法可以用于求解其行列式，相信到现在的话大家课上可能也已经见过了，这里提供一种用初等变换的方法

1. 把各行都加到第一行，然后提取公因子得到

$$\frac{n(n+1)}{2}\begin{vmatrix} 1 & 1 & 1 & \cdots & 1 \\ n & 1 & 2 & \cdots & n-1 \\ n-1 & n & 1 & \cdots & n-2 \\ \vdots & \vdots & \vdots & \cdots & \vdots \\ 2 & 3 & 4 & \cdots & 1 \end{vmatrix}$$

2. 将第一行除第一列以外元素消为0

$$\frac{n(n+1)}{2}\begin{vmatrix} 1 & 0 & 0 & \cdots & 0 \\ n & 1-n & 2-n & \cdots & -1 \\ n-1 & 1 & 2-n & \cdots & -1 \\ \vdots & \vdots & \vdots & \cdots & \vdots \\ 2 & 1 & 2 & \cdots & -1 \end{vmatrix}$$

然后按第一行展开，得到

$$\frac{n(n+1)}{2}\begin{vmatrix} 1-n & 2-n & \cdots & -1 \\ 1 & 2-n & \cdots & -1 \\ \vdots & \vdots & \cdots & \vdots \\ 1 & 2 & \cdots & -1 \end{vmatrix}$$

3. 从下往上，依次把上一行的 -1 倍加到下一行上去，第一行不变，可得

$$\frac{n(n+1)}{2}\begin{vmatrix} 1-n & 2-n & \cdots & -2 & -1 \\ n & 0 & \cdots & 0 & 0 \\ \vdots & \vdots & \cdots & \vdots & \vdots \\ 0 & 0 & \cdots & n & 0 \end{vmatrix}$$

此时就很显然了，可得最终结果是

$$\det = \frac{n^{n-1}(n+1)}{2}(-1)^{n+1}$$

加上 a 之后，核心是不变的，仍然是：把各行都加到第一行、提取公因子、将第一行除第一列以外元素消为0；此时就可以发现右下角那个 (n-1) 阶的部分是一样的，大家可以自己试一试

exp 如下

代码块

```
from Crypto.Util.number import long_to_bytes

n = 521
det = 
29695517968208859030649668195909804845925215827905038318044520938585868064869408320495166862663096808125759146766588276203415410350002487772917097754608566023524242916728640417169245095218950724342106675332097108073362922907378171277613194773846544274553875398735121489497742491419836910535609380722856494253185736028381520226148650020689726481311436899656399600765002365516451184621611467927855739837723913004608274187189326469932906359710882902516984538684668213593399202973983772391300460827418718932646993290635971088290251698453868466821359339920297
```

```
075930511169367613687545028064660955455207715570710861867657220435856253201368757497518241283929553547883533293654718717863009630601329406833110319404529947829512120759273820866492586665113686825362706954758222343169882716015867321595437344157979301799171937992927213403406355198319362445295406779529784609380742781174328239833920742705696157784907838373341938910253081911139595178753611035292322191383798679701517484126491018281034298331308609775635369403693102609463238394628293067325519216378890377212623531556936770983518634022216161093235126497361300977064080906643127187211141313038306929931570784734806087501780974144725354586628043568431029953492546635887696640949098694027257418071907603632634195812595541483400926410813964825764217759025463065958983692881825431236861934368161723367798103890886039175534263891168372978345126988876642853098813087652196853712173968242875697163541266206584355924340384369170421510970696838174208245750105548979276298700662358504142607607202450090670308682017711918547774618585863818302970468852253970353946009823259602084966816433280761645058
```

```
flag = (det // (pow(n, n - 1) * pow(-1, n + 1))) - (n + 1) // 2
print(long_to_bytes(flag))
# moectf{m4ny_m37h0d5_2_c4lcul473_7h3_det_0f_a_cycl1c_m47r1x}
```

moePoly1

出题人：hata　分数：150 pts

本题构造了一个多项式，其系数就是 flag 的各个字符，故目标就是要恢复出多项式的系数；具体而言，有 44 个系数，可以查询 44 次，是刚刚好的

这个题并没有什么密码学知识，出出来的初衷就是想让锻炼一下写交互脚本，写多长的脚本取决于你用什么方法和工具 (当然如果是 ai 完成这项任务的话那就无所谓了)

假设多项式系数分别为 $f_0, f_1, \cdots, f_{43}$ ，我们查询的 x 为 $x_1, x_2, ..., x_{44}$ ，分别得到了对应的 y，那么实际上就是得到了一个方程组，我们可以把它写成矩阵形式

$$\begin{pmatrix} x_1^0 & x_1^1 & \cdots & x_1^{43} \\ x_2^0 & x_2^1 & \cdots & x_2^{43} \\ \vdots & \vdots & \cdots & \vdots \\ x_{44}^0 & x_{44}^1 & \cdots & x_{44}^{43} \end{pmatrix} \begin{pmatrix} f_0 \\ f_1 \\ \vdots \\ f_{43} \end{pmatrix} \equiv \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_{44} \end{pmatrix} \pmod{p}$$

然后可以用 sagemath 内置函数求解，事实上 sage 里面也有内置的多项式插值函数，在后面 moePoly4 可以看到

代码块

```
from pwn import *

io = remote('127.0.0.1', 51293)
io.recvuntil(b'p = ')
p = int(io.recvline()[:-1].decode())
```

```
y = []
l = 44
for i in range(l):
    io.recvuntil(b'choice: ')
    io.sendline(b'1')
    io.recvuntil(b'x: ')
    io.sendline(str(i + 1).encode())
    y.append(int(io.recvline()[:-1].decode()))

A = matrix(Zmod(p), l, l)
for i in range(l):
    A[i,0] = 1
    for j in range(1, l):
        A[i,j] = pow(i + 1, j, p)
y = vector(Zmod(p), y)
ans = A.solve_right(y)
print(''.join([chr(i >> 248) for i in ans]))
```

moePoly3

出题人：hata　分数：150 pts

这个题查询次数更少了，需要一次恢复出 31 个系数；值得注意的是系数范围都在 0~255 之间，也就是说不会超过 8 bit

故这里的做法其实就是只需要传入 2^8 即 256，各系数的二进制表示就会恰好错开，连着排列在一起

因为 $31 * 8 = 248$，而 p 是 256bit，结果也不会超过 p，这时候只需要依次读出来即可

代码块

```
# y = 提交 256 得到 y
y_b = bin(y)[2:].zfill(248)
tmp = [str(int(y_b[i:i + 8], 2)) for i in range(0, len(y_b), 8)]
ans = ''.join([i for i in tmp[::-1]])
print(ans) # 提交 ans 即可得到 flag
```

这个题同样也没什么密码学知识点，算是个小游戏吧 🤖

XOR_revenge

出题人：youzi　分数：200 pts

本题为 `justXOR` 的加强版。由于 `enc` 是由 flag 和 key异或而来，所以我们只要求出key就能恢复flag

但问题在于n过大，对于一个 O(n) 的递推计算来说不可能算出来。本题和 `justXOR` 一样，需要改进这个递推算法

和 `justXOR` 不一样的是，这里设计了一个很复杂的递推公式，使得通项很难求：

- 初始值 $f_0, f_1, \ldots, f_6 = 1, 2, 3, 4, 5, 6, 7$;
- 对 $k \geq 7$:

$$f_k = 3f_{k-1} + 5f_{k-2} + 7f_{k-3} + 11f_{k-4} + 13f_{k-5} + 17f_{k-6} + 36f_{k-7} \pmod{M}, \quad M = 2^{521} - 1$$

我们采用一种称作**矩阵快速幂**的方法，首先把这个线性递推式化成矩阵乘法：

$$\begin{bmatrix} f_k \\ f_{k-1} \\ f_{k-2} \\ f_{k-3} \\ f_{k-4} \\ f_{k-5} \\ f_{k-6} \end{bmatrix} = \underbrace{\begin{bmatrix} 3 & 5 & 7 & 11 & 13 & 17 & 36 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}}_{C} \begin{bmatrix} f_{k-1} \\ f_{k-2} \\ f_{k-3} \\ f_{k-4} \\ f_{k-5} \\ f_{k-6} \\ f_{k-7} \end{bmatrix}$$

于是

$$\begin{bmatrix} f_n \\ f_{n-1} \\ \vdots \\ f_{n-6} \end{bmatrix} = C^{n-6} \begin{bmatrix} f_6 \\ f_5 \\ \vdots \\ f_0 \end{bmatrix}$$

对 C 做模 M 的矩阵快速幂即可。

关于矩阵快速幂的具体实现，可以参考矩阵 - OI Wiki

exp：

代码块

```
from Crypto.Util.number import long_to_bytes

M = 2**521 - 1
INIT = [1, 2, 3, 4, 5, 6, 7]

C = [[0]*7 for _ in range(7)]
C[0] = [3, 5, 7, 11, 13, 17, 36]
for i in range(1, 7):
    C[i][i-1] = 1

def mat_mult(A, B):
```

```
    n = len(A)
    return [[sum(A[i][k] * B[k][j] for k in range(n)) % M for j in range(n)]
            for i in range(n)]

def mat_pow(A, e):
    R = [[int(i == j) for j in range(7)] for i in range(7)]
    while e:
        if e & 1:
            R = mat_mult(R, A)
        A = mat_mult(A, A)
        e >>= 1
    return R

def fast_key(n):
    if n < 7:
        return INIT[n]
    P = mat_pow(C, n - 6)
    init = [7, 6, 5, 4, 3, 2, 1]
    return sum(P[0][j] * init[j] for j in range(7)) % M

key = fast_key(10**25)
kb = long_to_bytes(key)
enc =
b'lz\xb0T\xeb\x18\xec\xcfQ\x8bv#\x17t\xf0\xc7\xdaM\x87\xf5sr\xef\xd0\xfc3.\xdc%
\xaaf\xf4\xd1\xc8\xb7\xef'
print(bytes([x ^ y for x, y in zip(enc, kb)]))
```

补充： sagemath内置了矩阵快速幂：

代码块

```
Matrix(GF(M), C) ** (n-6)
```

墨箓验真

出题人：youzi　分数：200 pts

本题是一个哈希函数找碰撞的挑战，我们先分析一下这个哈希函数的计算规则：

第一步：输入先补齐到 6 的倍数，不足部分依次补 `1, 2, 3, ...`

代码块

```
def pad_digits(text):
    if not is_digit_string(text):
```

```
        raise ValueError("input must be a non-empty digit string")

    missing = (6 - len(text) % 6) % 6
    for value in range(1, missing + 1):
        text += str(value)
    return text
```

第二步：将填充后的输入按每6位进行分块 $B_j = (b_{j,1}, ..., b_{j,6})$,将其与6位（二进制位）的输入密钥进行计算

$$n_j = (-1)^{k_1} b_{j,1} + ... + (-1)^{k_6} b_{j,6}$$

代码块

```
def block_hash(block, key):
    if not isinstance(block, str) or len(block) != 6 or not block.isdigit():
        raise ValueError("block must be a 6-digit string")

    bits = normalize_key(key)
    total = 0

    for bit, digit in zip(bits, block):
        total += (-1 if bit else 1) * int(digit)

    return total
```

第三步：交替加减计算哈希

$$H = n_1 - n_2 + n_3 - n_4 + ...$$

代码块

```
def hash_with_key(text, key):
    padded = pad_digits(text)
    total = 0

    for index in range(0, len(padded), 6):
        sign = 1 if (index // 6) % 2 == 0 else -1
        total += sign * block_hash(padded[index : index + 6], key)

    return total
```

判断两个输入碰撞，首先这两个输入不能相同，然后就是它们计算出的哈希值一样。

代码块

```
def is_collision(original, candidate, key):
    if not is_digit_string(original) or not is_digit_string(candidate):
        return False
    if original == candidate:
        return False

    return hash_with_key(original, key) == hash_with_key(candidate, key)
```

题目分为三关：

1. 试墨台：提交与给定12位数碰撞的任意位的数

最简单的碰撞方法，就是直接在12位数后加入6个0，这6个0对哈希值显然没有任何贡献，因此产生碰撞。

2. 折简阁：提交与给定12位数碰撞的不超过7位的数

本关要求提交不超过7位，因此不能补0了。我们可以想办法找到一个通用碰撞，先计算：

$$A_i(P) = b_{1,i} - b_{2,i} + b_{3,i} - b_{4,i} + \cdots$$

注意到，原哈希可以改写成：

$$H(P,K) = \sum_{i=1}^{6} (-1)^{k_i} A_i(P)$$

所以只要满足 $A_i(Q) = A_i(P)$ 就对所有密钥 K 都有 $H(Q,K) = H(P,K)$。

回到本题，不妨设需要构造的碰撞为 Q = abcdefg ,填充后：

$$abcdef|g12345$$

计算 S(Q)：

$$S(Q) = (a-g, b-1, c-2, d-3, e-4, f-5)$$

同理可以在本地针对旧符 P 计算 $S(P) = (v_0, v_1, v_2, v_3, v_4, v_5)$, 令$S(Q) = S(P)$：

代码块

```
b = v1 + 1
c = v2 + 2
d = v3 + 3
e = v4 + 4
f = v5 + 5
a - g = v0
```

3. 霆纹坛：提交与给定18位数碰撞的不超过12位的数

直接构造 12 位两块短符即可：

```
代码块
1   0 = x1 x2 x3 x4 x5 x6 y1 y2 y3 y4 y5 y6
```

只需每列满足：

```
代码块
1   xi - yi = vi
```

moePoly2

出题人：hata　分数：250 pts

和 moePoly1 核心是一样的，唯一区别是查询次数少了 13 次。为什么可以变少呢？因为 flag 的内容并非都是未知的，其形如 `moectf{xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx}` 数一下发现 flag头，大括号，连字符，这些都是已知的，共有 12 个已知量，那我们只需要再爆破一个字符即可，因为 uuid 的格式，一个字符只有 16 种可能，枚举空间很小

> 😭 我最初的想法就是最后去爆破一个字符，当时并不知道 uuid.uuid4() 这东西有一位会是固定的 4，故完全没有考虑这一点。平台生成的动态 flag 虽然格式和 uuid4 是一致的，但是并不一定具备这一特征，而我将其混为一谈了
>
> 故若有师傅因此困扰，我非常抱歉，给大家跪了！！！

回到题目，同样假设多项式系数分别为 $f_0, f_1, \cdots, f_{43}$ (其中一部分是已知的)，我们查询的 x 为 $x_1, x_2, ..., x_{31}$ ，分别得到了对应的 y，为了方便分析，这次换个写法

$$f_0 \begin{pmatrix} x_1^0 \\ x_2^0 \\ \vdots \\ x_{31}^0 \end{pmatrix} + f_1 \begin{pmatrix} x_1^1 \\ x_2^1 \\ \vdots \\ x_{31}^1 \end{pmatrix} + \cdots + f_{43} \begin{pmatrix} x_1^{43} \\ x_2^{43} \\ \vdots \\ x_{31}^{43} \end{pmatrix} \equiv \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_{31} \end{pmatrix} \pmod{p}$$

之后只需要把 12 个已知的和枚举的一个 f_i 都移到右边，就可以得到一个方阵，此时解方程即可

这里提供一个我写的交互脚本

```
代码块
1   from pwn import *
2
3   io = remote('127.0.0.1', 61467)
4   io.recvuntil(b'p = ')
5   p = int(io.recvline()[:-1].decode())
6   y = []
```

```
l = 44 - 13
for i in range(l):
    io.recvuntil(b'choice: ')
    io.sendline(b'1')
    io.recvuntil(b'x: ')
    io.sendline(str(i + 1).encode())
    y.append(int(io.recvline()[:-1].decode()))

fakeflag = 'moectf{xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx}'
A = matrix(Zmod(p), l, l + 13)
y = vector(Zmod(p), y)
for i in range(l):
    A[i,0] = 1
    for j in range(1, l + 13):
        A[i,j] = pow(i + 1, j, p)

A1 = A
for i in range(len(fakeflag)):
    if fakeflag[i] != 'x':
        y -= (ord(fakeflag[i]) << 248) * A.column(i)

A1 = A1.delete_columns([0, 1, 2, 3, 4, 5, 6, 43, 15, 20, 25, 30, 7])
y1 = y
for i in '0123456789abcdef':
    y1 = y - (ord(i) << 248) * A.column(7)
    ans = A1.solve_right(y1)
    try:
        flag = i + ''.join([chr(i >> 248) for i in ans])
        if all(j in '0123456789abcdef' for j in flag):
            break
    except:
        print(i, 'wrong')
        pass
print('moectf{' + flag[:8] + '-' + flag[8:12] + '-' + flag[12:16] + '-' +
flag[16:20] + '-' + flag[20:] + '}')
```

JK_R0LL1ng

出题人：youzi　分数：300 pts

题目提供加密脚本和密钥流。密钥流有 128 个字节，也就是 1024 位。目标是恢复 flag 中的 14 位十六进制密钥。

密钥分成三段，分别作为三个 LFSR 的初态：前两段各 5 位十六进制数，最后一段 4 位。脚本要求每个初态的最高位为 1，所以状态范围分别是：

$$2^{16} \le R_1 < 2^{17}, \qquad 2^{18} \le R_2 < 2^{19}, \qquad 2^{15} \le R_3 < 2^{16}$$

三个寄存器的反馈掩码如下：

寄存器	位数	反馈掩码
R1	17	0x12000
R2	19	0x72000
R3	16	0xB400

每一步，程序先从三个寄存器取出反馈位 j、k、x，再用 j 和 k 更新 JK 触发器，最后输出一位密钥流。记第 n 步更新后的触发器状态为 q_n。初始状态与输出关系为：

$$q_0 = 1, \qquad z_n = q_n \oplus x_n$$

JK 触发器的状态更新可以写成：

$$q_n = \begin{cases} j_n, & q_{n-1} = 0, \\ 1 \oplus k_n, & q_{n-1} = 1. \end{cases}$$

所以，只要知道相邻两步的触发器状态，就能确定一个反馈位：

$$q_{n-1} = 0 \Longrightarrow j_n = q_n, \qquad q_{n-1} = 1 \Longrightarrow k_n = 1 \oplus q_n$$

先猜 R3，就能生成它的反馈序列 x_n，再由密钥流恢复触发器状态：

$$q_n = z_n \oplus x_n, \qquad 1 \le n \le 1024$$

这样，每一步都会为 R1 或 R2 提供一个已知反馈位。

LFSR 的反馈位是当前状态若干位的异或。状态更新由移位和异或构成，因此第 n 步的反馈位可以写成初态各位的线性组合。

设寄存器初态为 s，第 n 步对应的系数掩码为 v_n，则反馈位为：

$$b_n = \mathrm{parity}(s \,\&\, v_n), \qquad \mathrm{parity}(u) = wt(u) \bmod 2$$

$wt(u)$ 表示 u 的二进制表示中 1 的个数。若寄存器反馈掩码为 M，系数掩码按以下递推生成：

$$v_1 = M, \qquad v_{n+1} = \left\lfloor \frac{v_n}{2} \right\rfloor \oplus \begin{cases} M, & v_n \bmod 2 = 1, \\ 0, & v_n \bmod 2 = 0. \end{cases}$$

对每个候选 R3，先恢复 q_n，然后根据上一步的状态分别加入以下方程：

$$q_{n-1} = 0 \Longrightarrow \mathrm{parity}(R_1 \,\&\, v_{1,n}) = q_n,$$

$$q_{n-1} = 1 \Longrightarrow \mathrm{parity}(R_2 \,\&\, v_{2,n}) = 1 \oplus q_n.$$

这些方程都在二元域上。对 R1 和 R2 分别做高斯消元，就能求出两个寄存器的初态。

R3 是 16 位且最高位必须为 1，因此候选范围为：

$$2^{15} \le R_3 < 2^{16}$$

对每个候选，增量式高斯消元逐条处理方程。错误的 R3 通常很快会让方程组产生矛盾，可以立即舍弃。正确的 R3 会让两组方程相容，并且有足够多的独立方程唯一确定 R1、R2。

exp:

代码块

```
R1_SIZE, R2_SIZE, R3_SIZE = 17, 19, 16
R1_MASK, R2_MASK, R3_MASK = 0x12000, 0x72000, 0xB400
R1_FULL = (1 << R1_SIZE) - 1
R2_FULL = (1 << R2_SIZE) - 1
R3_FULL = (1 << R3_SIZE) - 1

ciphertext = bytes.fromhex(
    "6c99b6c6df18a2832288385028646f5e26cb123546fc17365d535a36337484e1"
    "e7feb9c3839cb2a60a413152dd0d78fc3de355d8f9a0c64d5623a4249d5aa264"
    "4f0fe9279b3156de5bc41d58e96fd0e7592e82087d24533c2a7319ee1de1492b"
    "4f80d47cc585ee7b12041b044c083b77d67c75938840f3a261def67d7120e0a2"
)
stream = [
    (byte >> bit) & 1
    for byte in ciphertext
    for bit in range(7, -1, -1)
]


def step(state, feedback_mask, state_mask):
    bit = (state & feedback_mask).bit_count() & 1
    return ((state << 1) & state_mask) | bit, bit


def coefficients(mask, count):
    result = []
    vector = mask
    for _ in range(count):
        result.append(vector)
        vector = (vector >> 1) ^ (mask if vector & 1 else 0)
    return result


class GF2:
    def __init__(self, size):
        self.rows = [None] * size
        self.rhs = [0] * size

    def add(self, row, value):
        while row:
            pivot = row.bit_length() - 1
```

```
            if self.rows[pivot] is None:
                self.rows[pivot] = row
                self.rhs[pivot] = value
                return True
            row ^= self.rows[pivot]
            value ^= self.rhs[pivot]
        return value == 0

    def solve(self):
        if any(row is None for row in self.rows):
            return None
        solution = 0
        for pivot, row in enumerate(self.rows):
            bit = self.rhs[pivot] ^ ((row & solution).bit_count() & 1)
            solution |= bit << pivot
        return solution


V1 = coefficients(R1_MASK, len(stream))
V2 = coefficients(R2_MASK, len(stream))

for r3_initial in range(0x8000, 0x10000):
    r1_system = GF2(R1_SIZE)
    r2_system = GF2(R2_SIZE)
    r3 = r3_initial
    q_before = 1
    consistent = True

    for n, z in enumerate(stream):
        r3, x = step(r3, R3_MASK, R3_FULL)
        q_after = z ^ x
        if q_before == 0:
            consistent = r1_system.add(V1[n], q_after)
        else:
            consistent = r2_system.add(V2[n], q_after ^ 1)
        if not consistent:
            break
        q_before = q_after

    if not consistent:
        continue

    r1 = r1_system.solve()
    r2 = r2_system.solve()
    if r1 is None or r2 is None:
        continue
    if not (0x10000 <= r1 < 0x20000 and 0x40000 <= r2 < 0x80000):
```

```
            continue

        key = f"{r1:05x}{r2:05x}{r3_initial:04x}"
        print(f"moectf{{{key}}}")
        break
```

ez_lattice

出题人：statwalking　分数：300pts

只是一个经典的 AGCD问题(the Approximate Common Divisor Problem)

我们回顾他的原始版，GCD问题，也就是最大公约数问题

我们知道一系列数 $x_1, x_2, ...x_n$,满足如下式子

$$
\begin{aligned}
x_1 &= pq_1 \\
x_2 &= pq_2 \\
&... \\
x_n &= pq_n
\end{aligned}
$$

GCD问题求得就是这个p，用欧几里得算法能很轻易的帮我们解决这个问题，那么AGCD问题是什么呢，就是在原有的基础上引入误差 $e_1, e_2, ..., e_n$,此时满足以下式子

$$
\begin{aligned}
x_1 &= pq_1 + e_1 \\
x_2 &= pq_2 + e_2 \\
&... \\
x_n &= pq_n + e_n
\end{aligned}
$$

在引入噪声的情况下，我们就难以求出这个p了，但在满足特定条件的情况下，我们可以通过格攻击去求取

我们选取任意一个对 (x_0, x_i) ，我们做如下式子

$$
\begin{aligned}
q_0x_i - q_ix_0 &= q_0(pq_i + e_i) - q_i(pq_0 + e_0) \\
&= pq_0q_i + q_0e_i - pq_0q_i - q_ie_0 \\
&= q_0e_i - q_ie_0
\end{aligned}
$$

依据这个关系，我们可以构造如下一个线性方程组

$$
(q_0, q_1, ...q_n) = \begin{bmatrix}
2^{\rho+1} & x_1 & x_2 & \cdots & x_n \\
 & -x_0 & & & \\
 & & -x_0 & & \\
 & & & \ddots & \\
 & & & & -x_0
\end{bmatrix} = (q_0 2^{\rho+1}, q_0e_1 - q_1e_0, q_0e_2 - q_2e_0, \cdots q_0e_n - q_ne_0)
$$

在本题的参数下，它就是格的短向量，我们取出这个短向量的第一个参数的绝对值，即为 $q_0 2^{\rho+1}$,因为规约出来可能是负值，而 ρ 是我们设定用于平衡的参数，已知，进而我们恢复了 q_0 ，那么由于 x_0

已知， e_0 小于 q_0 进而可以通过整除恢复 p

进而恢复所有消息，解题代码如下

代码块

```
from Crypto.Util.number import *
from sage.all import *

cipher=[402776039458631011297186546714867788098604202819,
373258092216104009527669023328944518921149130678 3,
692295900353209998824909093056129368939693425936 9,
802069204560407329734551954050630189069699467192 3,
692404214735338837211284576273644863038281796723 3,
373257845774142558165509992349876682515214014184 9]
k = 6
rho = 28

A = pow(2,rho+1)
B = Matrix(cipher[1:])
C = Matrix.diagonal([-cipher[0]]*(k-1))
N = Matrix.block([[A,B],[0,C]])
L = N.LLL()

a0 = (L[0,0] / A).abs()
M = cipher[0] / a0
msg = []
for c in cipher:
    msg.append(c // int(M))
secret = b''.join([long_to_bytes(m) for m in msg]).decode()
flag = f'moectf{{{secret}}}'
print(flag)
```

m1x3d_d1p

出题人：youzi 分数：300 pts

本题为三个离散对数问题(DLP)的叠加。

首先来看第二个dlp问题:

代码块

```
    p2, q2 = get_necessaryprime()
    n = p2*q2
    msg = bytes_to_int(salt)
    assert 0 < msg < n
```

```
5       c = pow(n - 1, msg, n^3)
```

要想恢复salt，必须得通过公开的 `n,c` 恢复 `msg` ，先观察一下这个式子：

$$c = (n-1)^m \mod n^3$$

显然这里可以使用二项式定理：

$$c = n^2 \cdot (-1)^{m-2}\binom{m}{2} + n \cdot (-1)^{m-1}\binom{m}{1} + (-1)^m \mod n^3$$

进一步改变模数：

$$c = mn \cdot (-1)^{m-1} + (-1)^m \mod n^2$$

$$c = (-1)^m \mod n$$

因此，可以先计算 $c \mod n$ 来判断 m 的奇偶性，然后可以从 $c \mod n^2$ 的结果中求出 m 。

然后我们再来看第三个离散对数问题，这是一个椭圆曲线上的离散对数问题(ECDLP)

代码块

```
1   p3 = 2^521 - 1
2   E = EllipticCurve(GF(p3), [1, 0])
3   assert p3.is_prime()
4   assert E.cardinality() == N
5   G = get_point(E)
6   Q = h*G     # ecc point multiplication
```

椭圆曲线为：

$$E/\mathbb{F}_{p_3} : y^2 = x^3 + x, \qquad p_3 = 2^{521} - 1$$

并明确给出了群阶： $\#E(\mathbb{F}_{p_3}) = 2^{521}$

题目输出了 Q=hG 和基点 G ，要求 h ($0 < h < p_1 < 2^{256}$)。

观察到，这个群阶光滑，它的所有素因子都等于2，适合使用**Pohlig–Hellman** 算法求离散对数

同时，由于 G 的阶一定整除 2^{521} ，而 $2^{520}G \neq \mathcal{O}$ ，可以确定

$$\mathrm{ord}(G) = 2^{521}$$

接下来我们来看一下算法的详细过程，大体可以视作从最低位起逐位求 $h = \sum_{i=0}^{520} b_i 2^i, \quad b_i \in \{0,1\}$

先构造一个点 $T = 2^{k-1}G$

因为 G 的阶是 2^k ，所以：

$$T \neq \mathcal{O}, \qquad 2T = \mathcal{O}$$

也就是说，T 的阶是 2。它的倍数只有两种结果：

$$hT = \begin{cases} \mathcal{O}, & h \text{ 是偶数}, \\ T, & h \text{ 是奇数}. \end{cases}$$

现在，把公开的 Q 乘上 2^{k-1}：

$$2^{k-1}Q = 2^{k-1}hG = hT$$

因此，我们只要计算这个点，就知道了 h 的奇偶，也就是它的最低位 b_0：

$$b_0 = \begin{cases} 0, & 2^{k-1}Q = \mathcal{O}, \\ 1, & 2^{k-1}Q = T. \end{cases}$$

已经知道最低位以后，可以写成：

$$h = b_0 + 2u$$

从 Q 中减掉已知部分：

$$R_1 = Q - b_0G = h - b_0G = 2uG$$

乘上 2^{k-2}：

$$2^{k-2}R_1 = 2^{k-2}2uG = uT$$

结果又只有 $\mathcal{O}$ 和 T 两种。它告诉我们 u 的奇偶，而 u 的最低位就是原来 h 的第二位 b_1。

继续恢复第三位时，先减掉前两位：

$$R_2 = Q - [b_0 + 2b_1]G$$

因为

$$h = b_0 + 2b_1 + 4v$$

所以

$$2^{k-3}R_2 = vT$$

比较结果与 $\mathcal{O}, T$，就得到第三位。后续每一位都按这个方式处理。

✅ 请注意，**Pohlig–Hellman** 算法支持的一般情况是，群阶可分解为多个小因子相乘（即群阶光滑），设群阶 n 最大的素因子为 p_m，则复杂度为 $O(\sqrt{p_m})$。具体的算法过程请自行搜索了解。

最后我们来看第一个离散对数问题

代码块

```
g = 4
x = LOWER + getRandomRange(0, int(WIDTH))
```

```
p1=get_safeprime()
h = pow(g, x, p1)
salt = get_random_bytes(16)
stream = shake_256(int_to_bytes(x, 16) + salt).digest(len(FLAG))
ct = bytes(xor(a, b) for a, b in zip(FLAG, stream))
```

我们已经知道

$$4^x = h \pmod{p_1}$$

题目生成的 p_1 为安全素数，突破口在 x 的生成方式：

$$x = 2^{72} + t, \qquad 0 \le t < 2^{50}$$

我们取 $H = h \cdot \left(4^{2^{72}}\right)^{-1} \bmod p_1$,则问题转换为求一个只有50位范围的离散对数：

$$4^t = H \pmod{p_1} \qquad 0 \le t < 2^{50}$$

可以使用 Pollard kangaroo，也可以使用 BSGS，两者的时间复杂度相同。下面简单介绍一下BSGS的过程：

令 $M = 2^{25}$ ，把 t 写成

$$t = iM + j, \qquad 0 \le i, j < M$$

那么

$$4^j = H \cdot (4^{-M})^i \pmod{p_1}$$

先建哈希表保存所有 baby steps 4^j ，然后枚举 giant steps $H(4^{-M})^i$ 。查到相等元素时，就得到 $t = iM + j$ 。由于 4 的阶 q1 大于整个搜索区间，区间内的解唯一。

✅ 实际上，**sagemath**已经实现了针对特定情况的DLP求解算法。包括 Pohlig–Hellman、BSGS、pollard rho、kangaroo。因此对于第一和第二个问题只需要直接调用discrete_log()即可

最后我们用求出的 x,salt 恢复密钥流并异或解密即可。

exp:

代码块

```
from hashlib import shake_256
from operator import xor
from sage.groups.generic import bsgs

p1 =
73874304220966794859883564142670914442160584279512562772063275425872997290543
```

```
n =
263484687903586619874452051338084813614357031304185138877058972788263087700338619238302749304366862787242542186163658141278956672012269466801651830087545475974748613034239106915516904449213861631685486145253812544059378499555071553140040075356040450612555536627278280512908323144496594847793087412937277079009001702356598622520736305530445263903011993170716798611654070769571461765619479900134841842021062002333990456417472344583077246616349736212821907925050461483256603141493961335092455593083177979856830060610328000238009611742123545449551095810649285784169719288072037495826690214585684528337665587709544386164971
c =
182922086004186806253461831567838421679802669087460635478747822895940160232907802063337543964503939943951020070358553440548041932173592064635634334464725057279949011329777637870802030269501693695915017874809576272590135679943579362083626876061014158639936980149866851190645338553720829300216132988903331455093924217167529953062964618684888329644153589843298829379319059034645591439822632682690834805877884208771765732977699681530591181118766250641994544866352256199440937218208211657057641652491551264648745509147130557486395900025636706210071063766163253781890841118695095223883101898855023378108102158752867981345035251074151425402482314130681643959794706262996227978812642092505007232833292192276701124661018465297026244111745515683869385539354779945365873065407814337682992342584415555457498111607618498863888370324716520477166805271122469370591350974397389633064200406471977025533580146342066813920822943716535160565901675716658362974018557863570301959874269498792461354825876433133611478573818000858641463048966351731460629472943074680706160423546376321119856390473404227057512503396261743449048606007215562580094926921313608736083766804758702927297596242355700336301750142340691233920910324415284976246137846298345495041805026543040610013738148314862314773649024857858110496374634593849900017966399133576556074130532388391960739491853686932053262887363865528698736655396222477448410577213518695026288501147596761924042499550735130522659899548174539064735577889051117478277148936358008701668007658607148425668305703944080877218631502661412261912265705652731553406304775772389290698575262636935771910020651106379322569915748479558639060723562806132708568861913765208374923601661009979851661498319996205442456377441636607257021328337239493969092073744626499647030361456047678810291701149086428927461595990547481722662454225710254539309758608593624
ct =
bytes.fromhex("c73ebfd0d5f5306bf7b8c259f87a866c40cbb5ce5f8d326174f09d179df3a5d3906d1e6fbf")

p3 = 2^521 - 1
E = EllipticCurve(GF(p3), [1, 0])
G =
E(6020778416720603517227477855097878740167361812994765487105307672032724334849113047243617528393766225587705066628296154430456880091295418013040021260550463027,
6918987005517118589053138410529597605888269407738916517383195558677289181938491309067198602979174232527906617161932894333247890172254269110529071110538439 78)
```

```
Q = E(212934006529485734624394736437801932354066420048603707451632233389231404515784375313870890678852497236882816821666511534557025609393615723436062288560049243 8, 181771705934197524062133755307139882791713557768164767172995686205928992875252359397822647532163829695795650196773270051797450973909960632877701333921095088 5)

sign = 1 if c % n == 1 else -1
u = sign * c % n^2
m = (-((u - 1) // n)) % n
salt = int(m).to_bytes(16, "big")

h = discrete_log(Q, G, ord=2^521, operation="+")

LOWER = 2^72
WIDTH = 2^50
g = Mod(4, p1)
target = Mod(h, p1) / (g^LOWER)
t = bsgs(g, target, (0, WIDTH - 1))
x = LOWER + t

stream = shake_256(int(x).to_bytes(16, "big") + salt).digest(len(ct))
flag = bytes(xor(a, b) for a, b in zip(ct, stream))
print(flag.decode())
```

西小电银行

出题人：youzi 分数：300 pts

本题的突破口在 CTR 的计数器分配方式：三条记录使用同一把 SM4 密钥，初始计数器也就是编号 `index` 又恰好连续，因此不同记录实际使用的密钥流发生了重叠。利用第二条记录的已知明密文，可以恢复第三条记录的银行卡号、PIN 和用户 ID 前缀，再用公开的哈希摘要爆破剩余的 4 位十六进制后缀。

SM4 的分组长度是 16 字节。CTR 模式先加密计数器，生成密钥流，再将密钥流与明文异或。设某条记录的初始计数器为 V，第 j 个分组从 0 开始编号，则：

$$S_j = E_k(V + j), \qquad C_j = P_j \oplus S_j$$

其中 E_k 表示用密钥 k 进行 SM4 分组加密。计数器递增的是完整的 128 位值，最后一个不足 16 字节的分组只使用相应长度的密钥流；CTR 不需要填充明文。

已知某段明文和对应密文，就能恢复该段密钥流：

$$S_j = C_j \oplus P_j$$

题目的初始计数器由 index 生成。`get_iv_byte()` 的生成逻辑为：

代码块

```
index_bytes = struct.pack(">I", index)
nonce = b"\x00" * 12 + index_bytes
```

也就是将序号编码为 4 字节大端整数，前面补 12 个零。三条记录共用同一把密钥，序号分别为 n、n+1、n+2，密钥流因此排列成：

代码块

```
                  第0个分组    第1个分组     第2个分组     第3个分组
record1 (n)       Ek(n)       Ek(n+1)      Ek(n+2)      Ek(n+3)    ...
record2 (n+1)     Ek(n+1)     Ek(n+2)      Ek(n+3)      Ek(n+4)    ...
record3 (n+2)     Ek(n+2)     Ek(n+3)      Ek(n+4)      Ek(n+5)    ...
```

因此，只要把第二条记录的明文和密文都跳过前 16 字节，再进行异或，就能得到第三条记录所需的密钥流：

$$P_3[i] = C_3[i] \oplus C_2[i+16] \oplus P_2[i+16]$$

上述方法能恢复的长度，是第二条记录的明文长度减去 16 字节。而三条记录的明文长度都为 146 字节，还差 `userid` 的4位后缀以及 `;risk=low;ok` 刚好16字节

爆破ID的4位后缀，把候选完整 ID 按题目代码中的方式计算摘要，比较前 12 个十六进制字符即可。

exp:

代码块

```
record2 = {}
ciphertext2 = ""
ciphertext3 = ""
plaintext2 = (
    "bank=XiXiaoDianBank;branch={branch};serial={serial};"
    "card={card};pin={pin};user_id={user_id};risk=low;ok"
).format(**record2).encode()

p3 = bytes(
    c ^ known_c ^ known_p
    for c, known_c, known_p in zip(
        bytes.fromhex(ciphertext3),
        bytes.fromhex(ciphertext2)[16:],
        plaintext2[16:],
    )
).decode()
print(p3)
```

代码块

```python
from hashlib import sha256

card = ""
pin = ""
user_id_prefix = ""
salt = ""
id_hash = ""

for value in range(0x10000):
    user_id = user_id_prefix + f"{value:04x}"
    digest = sha256(f"{salt}:{user_id}".encode()).hexdigest()[:12]
    if digest == id_hash:
        print(f"{card},{pin},{user_id}")
        break
else:
    raise ValueError("user_id suffix not found")
```

moeChamp10n

出题人：youzi　分数：400 pts

---

1. 题目定义

对任意整数 c，定义

$$w(c) = \min_{\substack{x,y\in\mathbb{Z}\\ ax+by=c}} (|x| + |y|)$$

满足以下四个条件的整数 c 称为本地冠军：

$$w(c) \geq w(c \pm a), \quad w(c) \geq w(c \pm b)$$

本题给出两个互素正整数 $a > b > 1$，要求找出整数上的所有“本地冠军”，将它们升序排列后哈希，生成 AES-GCM 密钥并解密 flag。

2. 证明过程

记 $ax + by = c$ 的整数解的解集为 $s(c)$

若 $(x_0, y_0) \in s(c)$ 且 $w(c) = |x_0| + |y_0|$，易知 $-a < y_0 < a$。否则不妨设 $y_0 \geq a$，有：

$$|x_0 + b| + |y_0 - a| \leq |x_0| + b + |y_0| - a < |x_0| + |y_0|$$

矛盾。

接下来我们先证明一个重要结论：

$$w(d) = \min\{|x| + y,\ |x + b| + a - y\} \qquad (1)$$

证明：由于 $(a,b)=1$,所以 $s(d)$ 里全部元素为

$$(x+tb,\ y-ta), \qquad t\in\mathbb{Z}$$

- 若 $t\le -1$ ，则 $y-ta\ge y+a>a$ ，不可能最短。
- 若 $t\ge 2$ ，则 $y-ta\le y-2a<-a$ ，也不可能最短。

因此只剩 t=0,1 ,证毕.

下设 c 是本地冠军

若 $y_0=0$ ，易得 $w(c)\ge w(c+b)$与$w(c)\ge w(c-b)$ 不能同时成立，故 $y_0\neq 0$

对于 $0<y_0<a$ 由 $(x_0,y_0-1)\in s(c-b)$知$w(c)>w(c-b)$ 成立

若 $x_0\ge 0,(x_0+b,y_0-a)\in s(c),(x_0+1+b,y_0-a)\in s(c+a),(x_0+1,y_0)\in s(c+a)$

由 (1) 可知， $w(c+a)=min\{x_0+1+b+a-y_0,x_0+1+y_0\}$

由 $w(c)$ 的最小性以及 $w(c)\ge w(c+a)$,有

$$x_0+b+a-y_0\ge x_0+y_0\ge x_0+1+b+a-y_0$$

矛盾，故 $x_0<0$

若 $x_0+b\le 0$ ，由 (1) 可知，

$$w(c-a)=min\{|x_0-1|+y_0,|x_0-1+b|+a-y_0\} \qquad (2)$$

由 (2), $w(c)$ 的定义及 $w(c)\ge w(c-a)$

$$-(x_0+b)+a-y_0\ge -x_0+y_0\ge min\{1-x_0+y_0,-(x_0+b-1)+a-y_0\}$$

也矛盾，那就确定了范围 $1-b\le x_0\le -1$ ，又因为 $(x_0+1,y_0)\in s(c+a)$

所以此时 $w(c)=-x_0+y_0>-(x_0+1)+y_0\ge w(c+a)$

由 (2), $w(c)\ge w(c-a)\iff$

$$-x_0+y_0\ge |x_0-1+b|+|y_0-a|=x_0+b-1+a-y_0\iff$$

$$y_0\ge \frac{a+b-1}{2}+x_0$$

同理， $w(c)\ge w(c+b)\iff$

$$-x_0+y_0\ge |x_0+b|+|y_0+1-a|=x_0+b+a-y_0-1\iff$$

$$y_0\ge \frac{a+b-1}{2}+x_0$$

由 $(x_0+b,y_0-a)\in s(c)$ ，所以 $-x_0+y_0\le x_0+b+a-y_0$ ，即

$$y_0\le \frac{a+b}{2}+x_0$$

所以

$$\frac{a+b-1}{2}+x_0\le y_0\le \frac{a+b}{2}+x_0$$

当 $2 \mid a+b$ 时，对每一个 $1-b \le x_0 \le -1$ 有唯一的 $y_0 = \frac{a+b}{2} + x_0$ ，并注意到此时 $|x_0|+|y_0| = |x_0+b|+|y_0-a|$ ，本地冠军 c 有 $b-1$ 个， $c = \frac{a+b}{2}(2x_0+b)$

当 $2 \nmid a+b$ 时，对每一个 $1-b \le x_0 \le -1$ 有唯一的 $y_0 = \frac{a+b-1}{2} + x_0$ ，并注意到此时 $|x_0|+|y_0| \neq |x_0+b|+|y_0-a|$ ，根据对称性，本地冠军 c 有 $2(b-1)$ 个，$c = \pm[(a+b)x_0 + \frac{a+b-1}{2}b]$

🧐 关于这里对称性的解释，就是考虑 $-a < y_1 < 0$ 的情况，可以取 $a(-x_1)+b(-y_1) = -c$ 这时 $0 < -y_1 < a$ ，同时不难证明 $w(c) = w(-c)$ 且 $-c$ 也是本地冠军，然后可以进行同理分析。但要注意可能存在 y_1 对应的 $-c$ 等于某个 y_0 对应的 c ，这样会重复计算。显然这里可以得出 $1 \le x_1 \le b-1$ ，因此 (x_1, y_1) 和 (x_0, y_0) 是一一对应取负值。实际上，能计算出 c 的 (x_1, y_1) 只可能有 (x_0+b, y_0-a) 。当 $w(c) = |x_0+b|+|y_0-a|$ 时，对于每一个 (x_0, y_0)，存在 $(x_1, y_1) = (x_0+b, y_0-a)$ ，进而 $ax_1+by_1 = a(-x_0)+b(-y_0) = c$ ，使得每一个 c 都被 -c 重复计算了。

3. 解题脚本

代码块

```python
a = 16302871846448256339569448006931389858269118713881858000841721100072112275228016532331388554188928492783936143483799084176586471963896405862124526512264974
b = 1097861

from hashlib import sha256
from typing import Iterable

iv = bytes.fromhex("ff8e74d7df70b1b968d0d68c")
ct = bytes.fromhex(

"f358dc96e2977fb91453ff381de25c4990e9436a0114cd77a593d66090f9254381baaf8cfd074683c3"
)
tg = bytes.fromhex("93ba6253a13e87c3174141f40b2cac11")

def derive_key(local_champions: Iterable[int]) -> bytes:
    h = sha256()
    for c in sorted(local_champions):
        h.update(str(c).encode())
        h.update(b"\n")
    return h.digest()
```

```
19
20  def decrypt_flag(key: bytes) -> bytes:
21      from Crypto.Cipher import AES
22
23      e = AES.new(key, AES.MODE_GCM, nonce=iv)
24      return e.decrypt_and_verify(ct, tg)
25
26  N = a + b
27  L = N // 2
28  champions = [N * p - b * L for p in range(1, b)]
29  if N % 2:
30      champions += [-c for c in champions]
31
32  key = derive_key(champions)
33  print(decrypt_flag(key).decode())
```

moePoly4

出题人：hata　分数：400 pts

这个题的任务是需要恢复 lcg 的种子，而 lcg 生成的序列是多项式的系数。那么很显然是需要两步来完成：先恢复系数，再求出种子

1. **恢复系数**

有 26 个系数，但是只能查询 20次，想要都恢复出来次数是不够的。为什么是 20 次，这里需要注意到 $20|p-1$

然后就是一个小技巧，找到模 p 乘法群中阶为 20 的子群的生成元，记为 g，即 $g^{20} \equiv 1 \pmod{p}$

接下来发送这样 20 个数 $g^0, g^1, \ldots, g^{19}$ ，其作用是：对于原始的多项式

$$f(x) = c_0 + c_1 x + c_2 x^2 + \cdots + c_{22} x^{22} + c_{23} x^{23} + c_{24} x^{24} + c_{25} x^{25}$$

大于等于 20 次的部分会降下来，例如

$$x^{20} = (g^i)^{20} = (g^{20})^i = 1$$
$$x^{21} = (g^i)^{21} = (g^{21})^i = g^i = x$$

故实际上得到了一个新的多项式

$$f(x) = (c_0 + c_{20}) + (c_1 + c_{21})x + (c_2 + c_{22})x^2 + \cdots + c_{19} x^{19}$$

这时有 20 个系数，除去前面六个，后面 14 个系数都是可以用的

至于如何找 g，可以先用 sage 内置函数找一个原根 g_0 ，然后 $g_0^{\frac{p-1}{20}}$ 即可

2. **恢复种子**

这里其实就是一个参数已知的截断 lcg 的板子题，给的是序列的高位信息，考的是 HNP，需要构造格；HNP 的构造对新生可能确实有点难度，不过这个东西搜一搜是可以搜出来一些文章的，里面也有板子可以用

lcg 的公式：$s_{i+1} \equiv as_i + b \pmod n$

改写成高低位的形式：

$$h_{i+1} + l_{i+1} \equiv a(h_i + l_i) + b \pmod n$$
$$l_{i+1} = al_i + ah_i + b - h_{i+1} \pmod n$$

已知部分记为 $t_i \equiv ah_i + b - h_{i+1} \pmod n$，则 $l_{i+1} \equiv al_i + t_i \pmod n$

往 HNP 目标形式上靠 $r_i = A_i u + B_i \pmod n$，其中，Ai 和 Bi 均已知，n 也已知，u 未知但是定值，ri 未知

现在很接近了，但是 l_i 不是定值，故还需进一步变形，考虑都往 l_0 上统一，写几项观察一下

$$l_1 \equiv al_0 + t_0 \pmod n$$
$$l_2 \equiv al_1 + t_1 \equiv a^2 l_0 + at_0 + t_1 \pmod n$$
$$l_3 \equiv al_2 + t_2 \equiv a^3 l_0 + a^2 t_0 + at_1 + t_2 \pmod n$$

故 $l_i \equiv A_i l_0 + B_i \pmod n (i \geq 1)$ (i=0 没意义)，其中

$$A_i \equiv a^i \pmod n$$
$$B_i \equiv aB_{i-1} + t_{i-1} \pmod n (i \geq 2)$$
$$B_1 = t_0$$

最后构造标准的 HNP 格，取 K=1 即可

$$(k_1, k_2, \ldots, k_v, l_0, 1) \begin{pmatrix} n & 0 & \cdots & 0 & 0 & 0 \\ 0 & n & \cdots & 0 & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & \cdots & n & 0 & 0 \\ A_1 & A_2 & \cdots & A_v & 1 & 0 \\ B_1 & B_2 & \cdots & B_v & 0 & K \end{pmatrix} = (l_1, l_2, \ldots, l_v, l_0, K)$$

参考 exp 如下

代码块

```
from pwn import *

io = remote('127.0.0.1', 51483)
io.recvuntil(b'a = ')
a = int(io.recvline()[:-1].decode())
io.recvuntil(b'b = ')
b = int(io.recvline()[:-1].decode())
io.recvuntil(b'p = ')
p = int(io.recvline()[:-1].decode())

g0 = primitive_root(p)
```

```
g = pow(g0, (p - 1) // 20, p)
R.<x> = PolynomialRing(GF(p))
points = []
for i in range(20):
    io.recvuntil(b'choice: ')
    io.sendline(b'1')
    io.recvuntil(b'x: ')
    tmp_x = pow(g, i, p)
    io.sendline(str(tmp_x).encode())
    y = int(io.recvline()[:-1].decode())
    points.append((tmp_x, y))
ans = list(R.lagrange_polynomial(points)) # 系数列表：低次->高次
output = ans[6:]

#######
n = p
h = [i * pow(2, 128) for i in output]

l = len(h)
t, A, B = [], [], []
for i in range(l - 1):
    tmp = a * h[i] + b - h[i + 1]
    t.append(tmp % n)
for i in range(1, l):
    A.append(pow(a, i, n))
B.append(t[0])
for i in range(1, l - 1):
    B.append((a * B[-1] + t[i]) % n)

L = matrix(ZZ, l + 1, l + 1)
for i in range(l - 1):
    L[i,i] = n
    L[-2,i] = A[i]
    L[-1,i] = B[i]
L[-2,-2] = 1
L[-1,-1] = 1

print(L.LLL()[0])
c6 = h[0] + L.LLL()[0][-2]
print(c6)
tmp = c6
for _ in range(7):
    tmp = (tmp - b) * pow(a, -1, n) % n
print(tmp)
io.recvuntil(b'seed: ')
io.sendline(str(tmp).encode())
print(io.recvline())
```

```
print(io.recvline())
```